

AZ500 Lab 8-9 Azure
Monitoring, Microsoft Defender
for cloud

Table of Contents

Introduction.....	2
Lab 8 Azure Monitor.....	2
Logging in as admin and downloading relevant files	2
Exercise 1: Deploying an Azure virtual machine	5
Task 1: Deploying an Azure virtual machine.....	5
Exercise 2: Creating a Log Analytics Workspace	9
Task 1: Creating a Log Analytics Workspace	9
Exercise 3: Creating an Azure Storage Account	11
Task 1: Creating an Azure Storage Account	11
Exercise 4: Creating a Data Collection Rule	13
Task 1: Creating a Data Collection Rule.....	13
Lab 09: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers	17
Exercise 1: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers	17

Introduction

In these labs, I gained hands-on experience with a range of advanced Azure security capabilities focused on protecting virtual machines and enhancing visibility across cloud environments. I began by deploying a virtual machine in Azure and implementing baseline security measures using Microsoft Defender for Cloud. This included enabling enhanced server protection features and configuring just-in-time (JIT) access to minimize exposure to external threats. I then progressed to integrating Microsoft Sentinel, where I onboarded data connectors, configured analytics rules, created playbooks for automated response, and tested custom alerts tied to suspicious activity. Each lab built upon the last, helping me understand how to layer proactive security, monitoring, and automation to better secure Azure workloads.

Lab 8 Azure Monitor

Logging in as admin and downloading relevant files

After logging into the SEA-Dev virtual machine, I began the lab by initiating the secure login process. From the remote desktop interface, I used the Ctrl + Alt + Delete command to bring up the Windows logon screen. This was accomplished either through the keyboard shortcut or by selecting the command from the lightning bolt icon located in the upper-left corner of the remote session interface. Once the login screen appeared, I signed in using the administrator credentials provided: the username “Admin” and the password “Pa55w.rd.”

The next step was to prepare the system by downloading the lab files necessary for the Azure Monitor exercises. A preconfigured script or shortcut was available on the desktop to automate this process. Running the script triggered a background task that cloned the latest lab resources from the official GitHub repository. This setup not only ensured that the most up-to-date files were used but also organized the directories required for the subsequent monitoring configurations. Upon completion, a confirmation message—“Success!”—was displayed, indicating that the lab environment had been properly initialized and was ready for use.

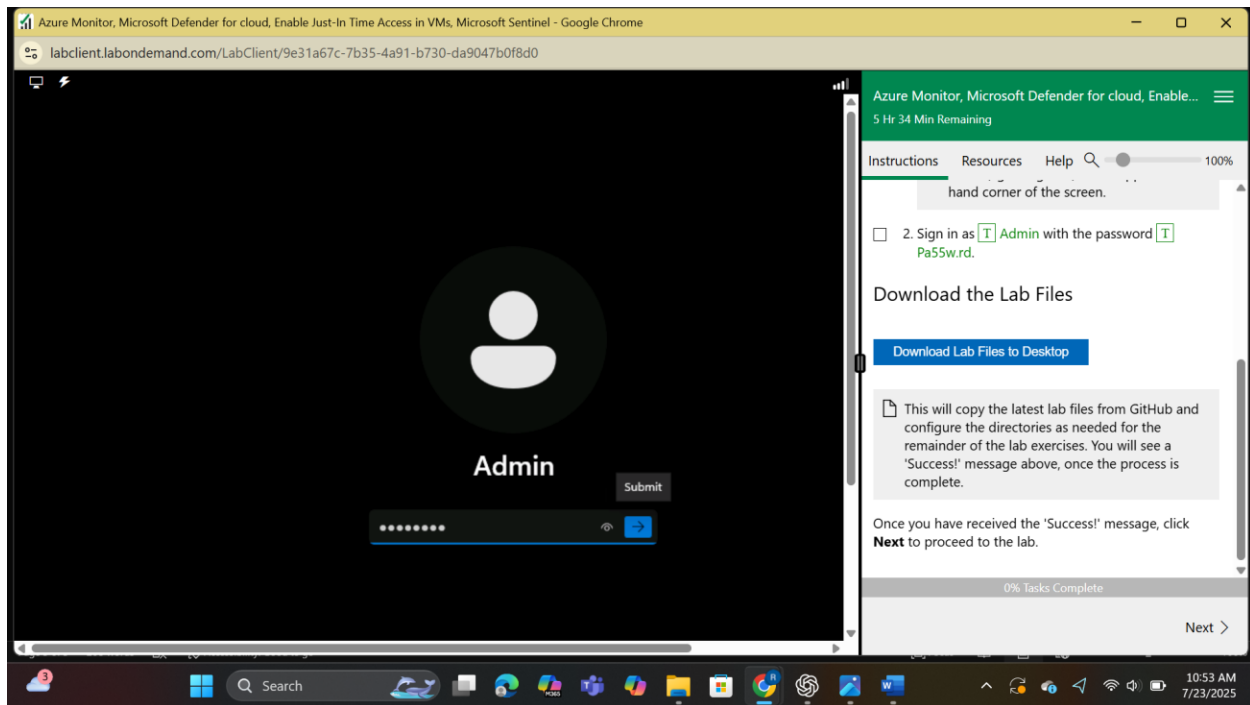


Fig 1.1 Logging in and downloading relevant files

After preparing the virtual machine environment and downloading the required lab files, I proceeded to access the Azure portal using the provided credentials. I navigated to <https://portal.azure.com> and signed in with the lab-specific user account: **LabUser-53202036@LODSPRODMCA.onmicrosoft.com**, using the password **M+6a!jF2Jp3n**. These credentials were supplied as part of the lab resources and were also accessible through the Resources tab in the Lab UI.

Upon successful login, I gained access to the pre-provisioned Azure subscription allocated for this lab session. This environment was configured to support all the tasks related to Azure Monitor, including resource deployment, diagnostic settings, and integration with monitoring services. Logging into the Azure portal marked the transition from local preparation to hands-on interaction with cloud resources, setting the stage for configuring and exploring Azure's native monitoring capabilities.

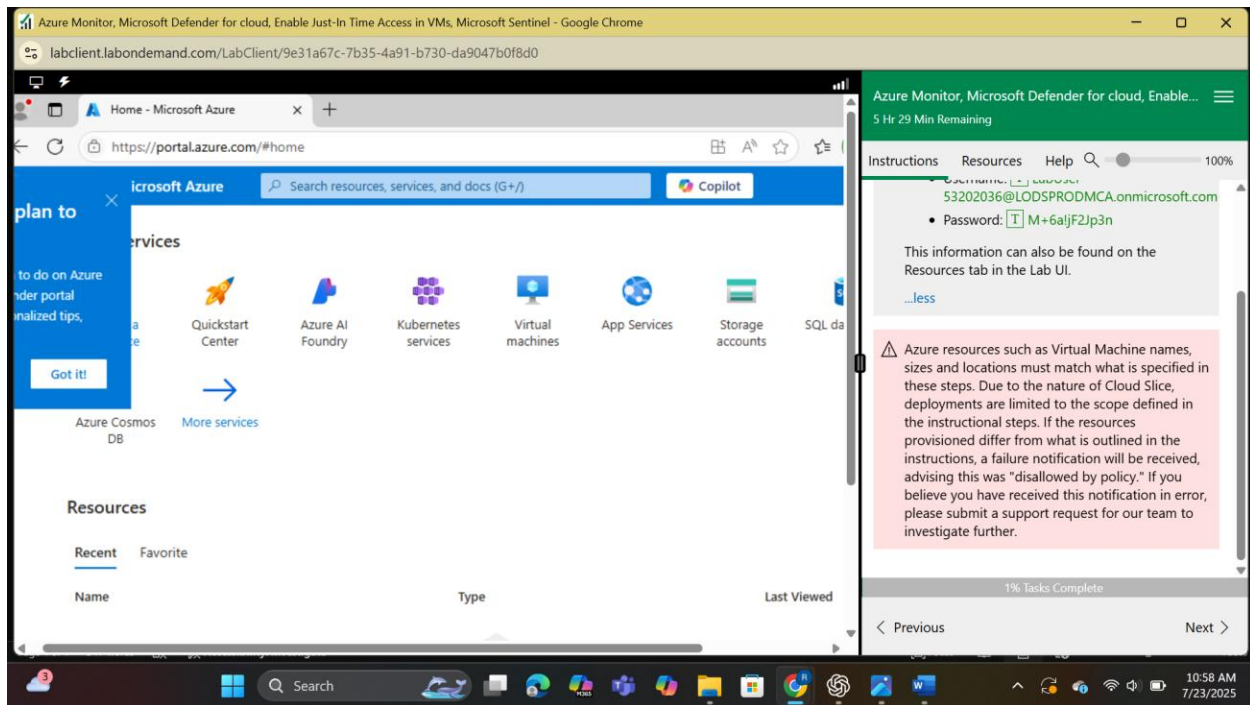


Fig 1.2 Successfully logging in to azure

After successfully signing in to the Azure portal with the provided lab credentials, I initiated the configuration steps necessary to ensure compatibility with the lab exercises. As noted in the lab guide, there is a known issue affecting the **New-AzVm** command when using **Az.Compute Module Version 4.24** in Azure Cloud Shell. This bug can disrupt the virtual machine deployment process, which is central to several tasks in the Azure Monitor lab.

To address this, I launched **Azure Cloud Shell** directly from the portal and executed a specific PowerShell command to install a stable module version. The command used was:

```
Install-Module Az.Compute -RequiredVersion 4.23.0 -Force
```

This ensured that the shell session utilized **Az.Compute Version 4.23.0**, which is unaffected by the bug and fully compatible with the instructions provided. The installation completed without error, allowing me to continue the lab using a reliable and tested module version for virtual machine operations.

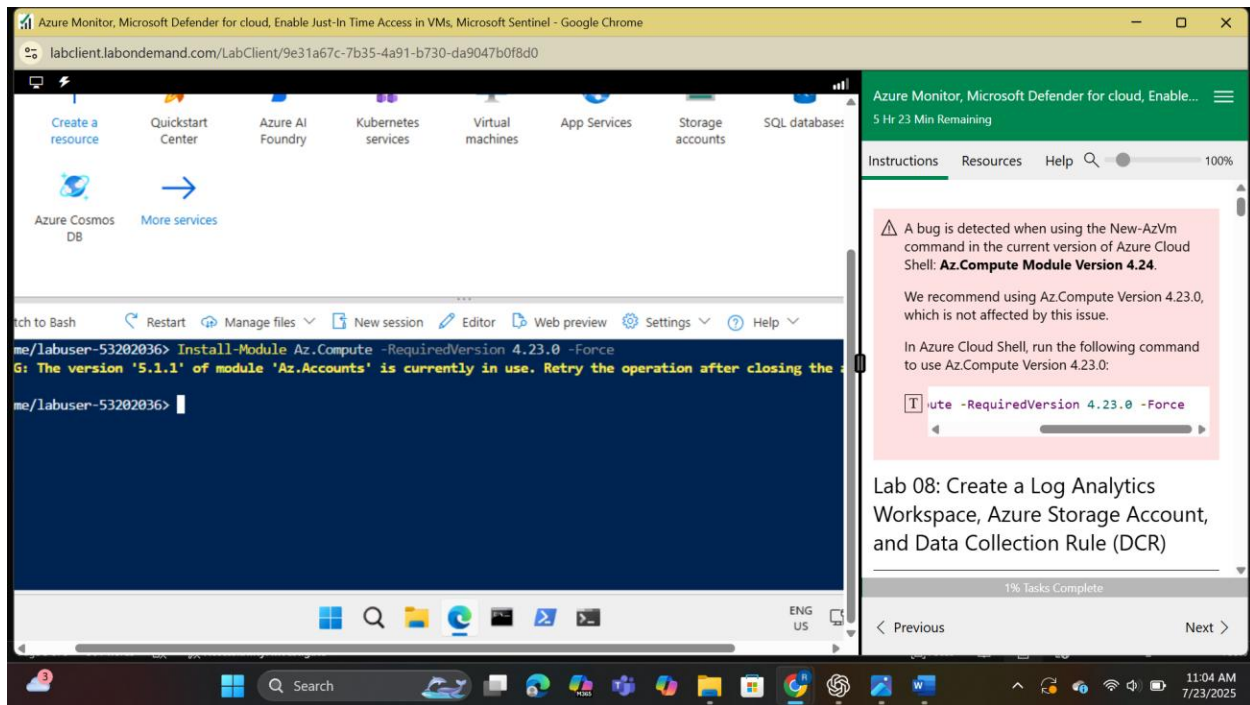


Fig 1.3 Downloading Az.Compute Version 4.23.0

Exercise 1: Deploying an Azure virtual machine

Task 1: Deploying an Azure virtual machine

After signing into the Azure portal at <https://portal.azure.com> using the provided credentials—**LabUser-53202036@LODSPRODMCA.onmicrosoft.com** with the password **M+6a!jF2Jp3n**—I began the process of deploying a virtual machine as part of Exercise 1. This step was foundational for the monitoring and data collection configuration that would follow in later exercises.

I first opened the **Cloud Shell** by selecting the terminal icon in the top-right corner of the Azure portal interface. When prompted, I ensured that **PowerShell** was selected as the scripting environment. In the “Getting Started” window, I left the default configuration intact and chose to proceed without mounting any additional storage accounts, as persistent storage was not required for this particular lab session. From the drop-down menu, I selected the appropriate subscription associated with my lab environment—**lodsubscription**—and then clicked **Apply** to initialize the shell.

Once the Cloud Shell environment was ready, I started by creating a new resource group that would serve as the logical container for all resources used in Labs 8, 9, and 10. To do this, I ran the following PowerShell command directly in the shell:

```
New-AzResourceGroup -Name AZ500LAB131415 -Location 'EastUS'
```

This command provisioned a resource group named **AZ500LAB131415** in the **East US** region, which had been predesignated as the standard region for all lab resources. The command completed successfully, confirming that the resource group was in place and ready to host all subsequent components, including the virtual machine, Log Analytics workspace, and storage account.

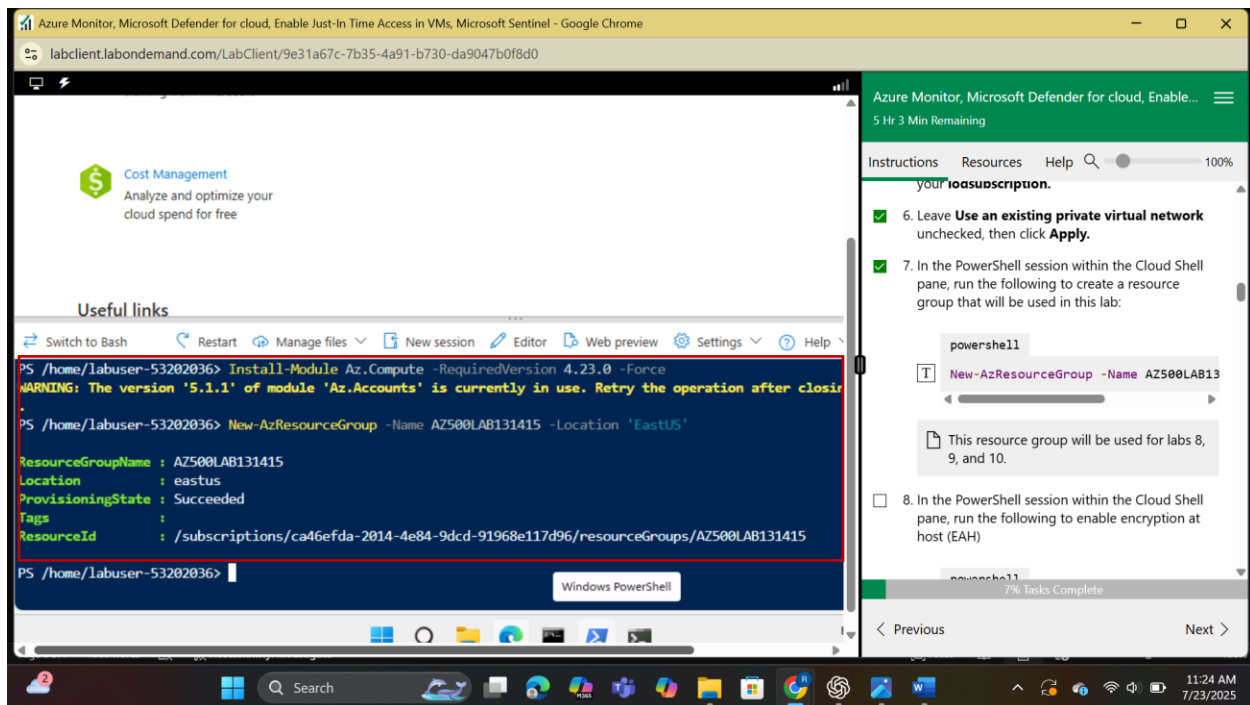


Fig 1.4 Successfully creating the lab resource

Next, I enabled a critical compute feature called **Encryption at Host (EAH)**. This feature ensures that data stored on the host machine running the virtual machine is encrypted at rest, thereby providing an additional layer of security beyond traditional disk-level encryption. To register this feature, I executed the following command:

```
Register-AzProviderFeature -FeatureName "EncryptionAtHost" -ProviderNamespace Microsoft.Compute
```

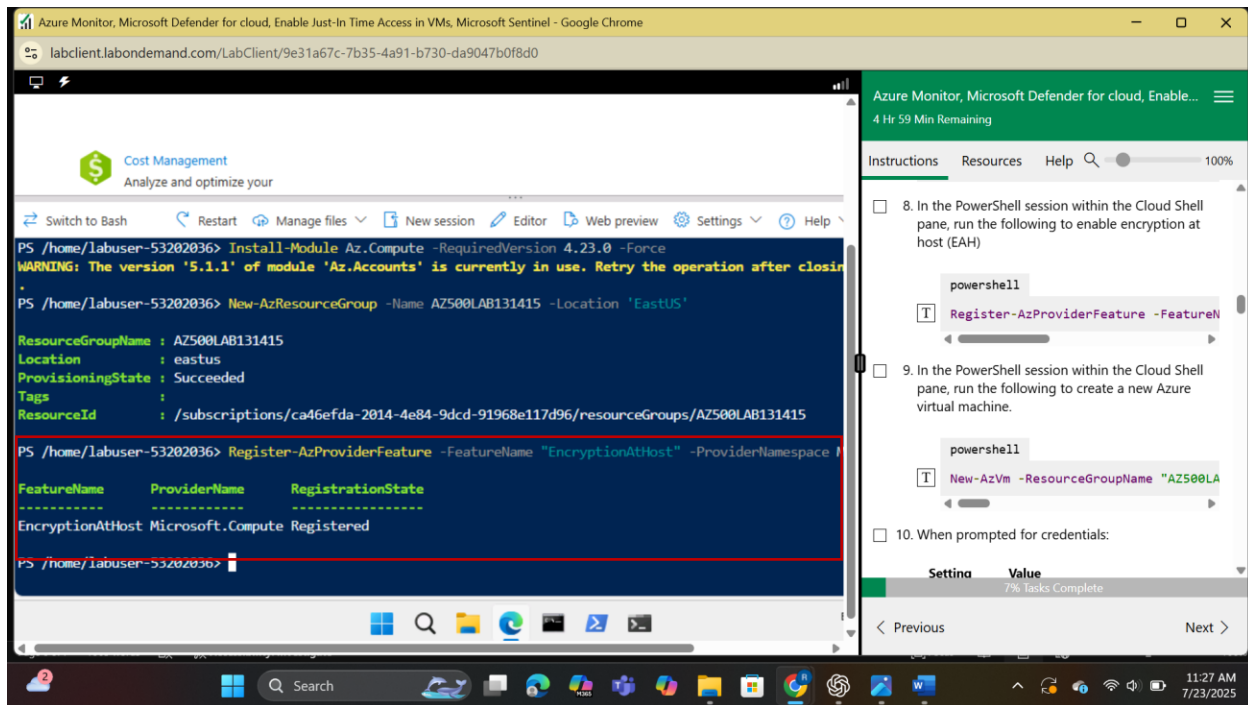


Fig 1.5 Enabling encryption at host (EAH)

Once the feature registration was initiated, I proceeded to deploy the virtual machine itself. I carefully constructed the VM deployment command, specifying all required parameters such as resource group, virtual network, subnet, public IP address, and open ports. The full command used to deploy the VM was:

```
New-AzVm -ResourceGroupName "AZ500LAB131415" -Name "myVM" -Location 'EastUS' -
VirtualNetworkName "myVnet" -SubnetName "mySubnet" -SecurityGroupName
"myNetworkSecurityGroup" -PublicIpAddressName "myPublicIpAddress" -PublicIpSku
Standard -OpenPorts 80,3389 -Size Standard_D2s_v3
```

Upon execution, I was prompted to enter credentials for the new virtual machine. I entered the username **localadmin** and used the custom password I had previously created in Lab 02, ensuring consistency with earlier configurations. The deployment process took a few minutes to complete, during which all the specified components—virtual network, subnet, network security group, public IP, and the virtual machine—were provisioned and linked together under the same resource group.

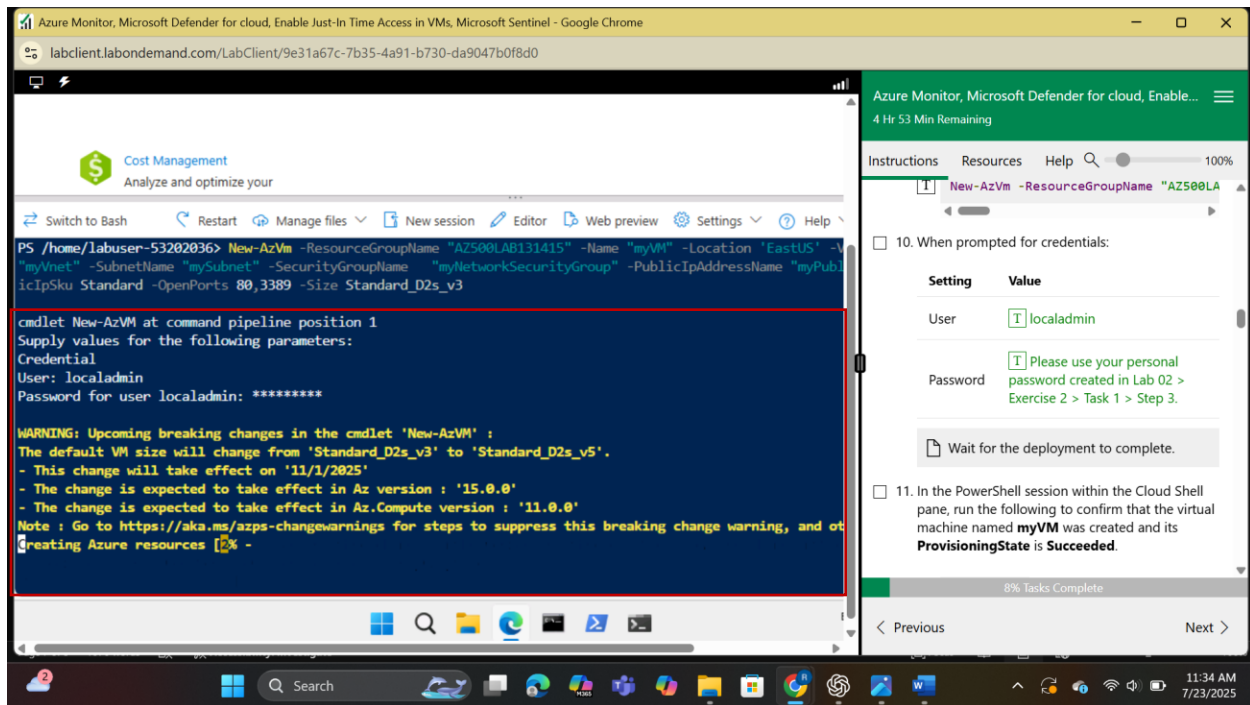


Fig 1.6 Creating a new Azure virtual machine.

To confirm that the VM deployment was successful, I ran the following verification command:

```
Get-AzVm -Name 'myVM' -ResourceGroupName 'AZ500LAB131415' | Format-Table
```

This command returned a formatted output showing the details of the virtual machine, including its provisioning state. I verified that the **ProvisioningState** was set to **Succeeded**, which confirmed that the VM was fully operational and ready for use in the next steps of the lab.

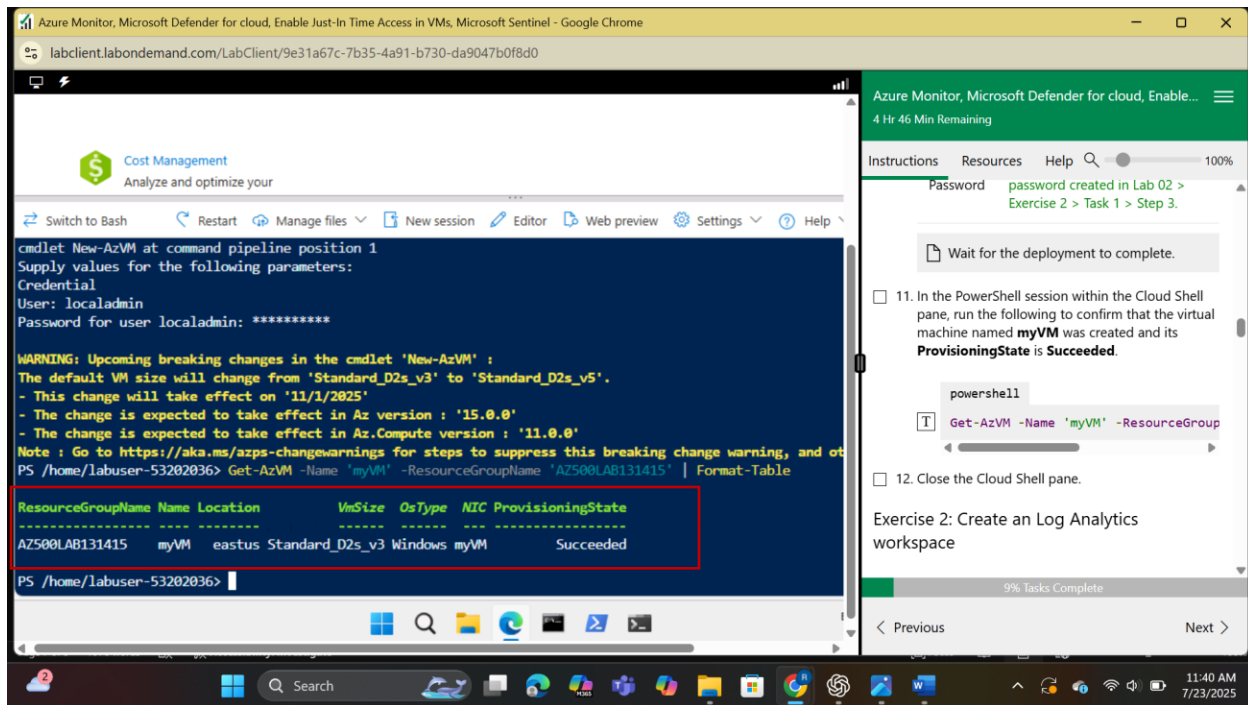


Fig 1.7 Confirming the deployment of the vm

After confirming the deployment, I closed the Cloud Shell pane, completing the first exercise with a clean and fully functional virtual machine environment ready for monitoring and data collection integration.

Exercise 2: Creating a Log Analytics Workspace

Task 1: Creating a Log Analytics Workspace

With the virtual machine successfully deployed and verified, I moved on to the next objective—creating a Log Analytics workspace that would serve as the central repository for monitoring data. This workspace is a key component in the Azure Monitor architecture, enabling the collection, analysis, and querying of log and performance data from connected Azure resources.

To begin, I navigated to the **Azure portal homepage** and used the global search bar at the top of the screen to search for “**Log Analytics workspaces**.” Once the service blade loaded, I selected the + **Create** button to initiate the workspace creation process.

On the **Basics** tab of the “Create Log Analytics workspace” blade, I specified the required configuration values for the workspace. I selected the appropriate **subscription** associated with my lab environment, and then selected the previously created resource group, **AZ500LAB131415**, to keep all resources logically grouped. For the workspace name, I entered **LAW53202036**, ensuring it met the naming standards. I also set the **Region** field to **East US**, aligning with the region used throughout the lab to avoid any cross-region dependency issues.

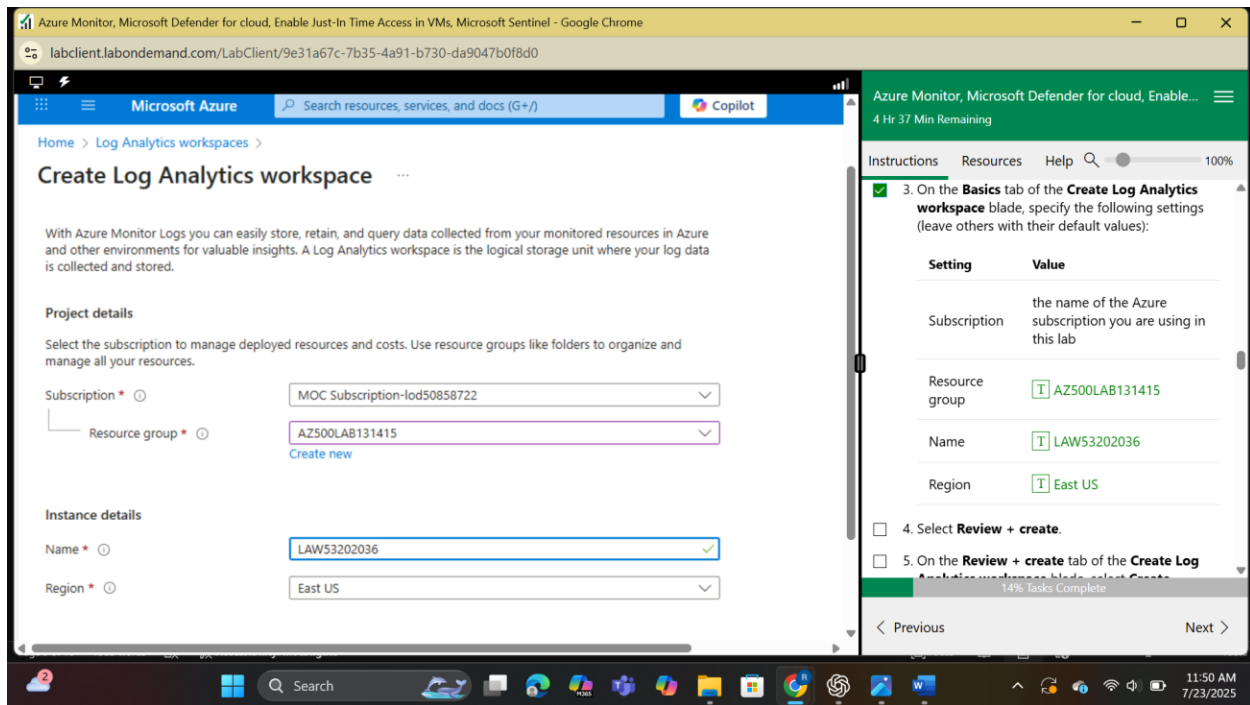


Fig 1.8 Filling in the relevant information on the Basics tab of the Create Log Analytics workspace blade

After entering the values, I clicked **Review + create**, and once the validation passed, I proceeded by selecting the **Create** button. The deployment process was initiated and completed within a short time, after which I was able to confirm that the workspace **LAW53202036** appeared in the list of available Log Analytics workspaces under the correct subscription and resource group.

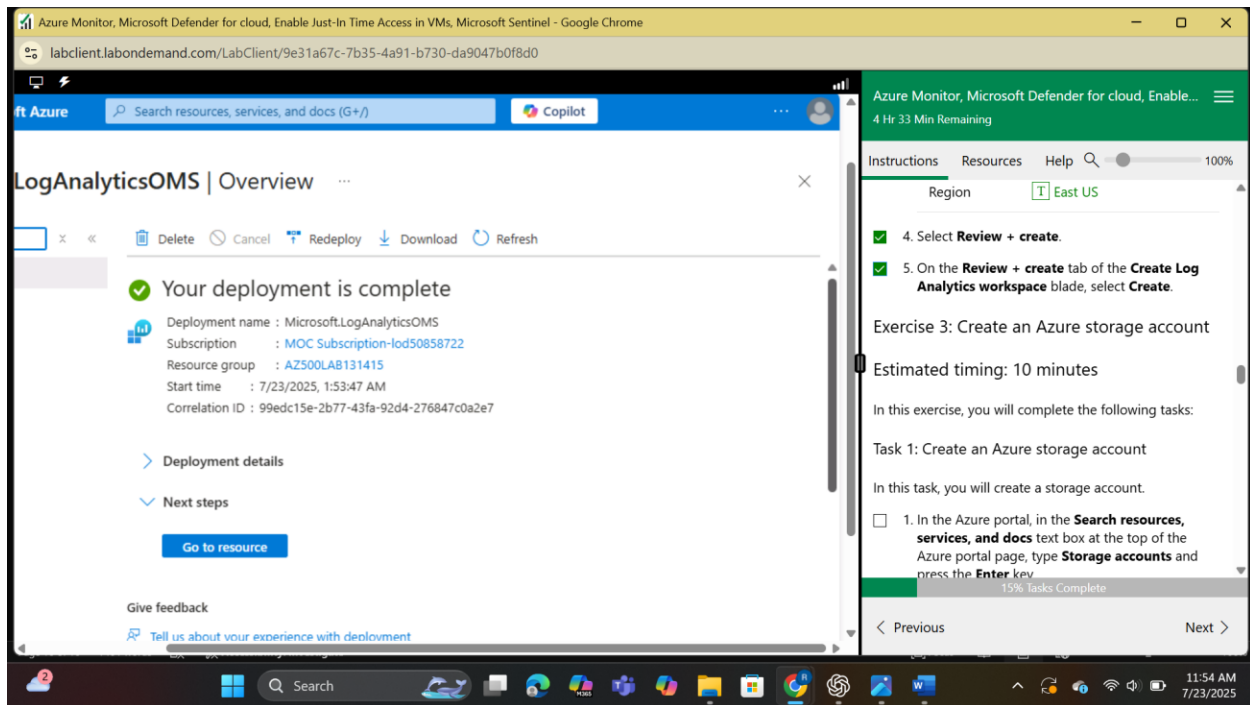


Fig 1.9 Successful deployment of the log analytic work space

With the workspace now in place, I had successfully established the foundational element required for log ingestion and data collection in Azure Monitor. This workspace would later be linked to data collection rules and virtual machines to gather metrics and logs essential for visibility, threat detection, and performance tuning.

Exercise 3: Creating an Azure Storage Account

Task 1: Creating an Azure Storage Account

Following the successful creation of the Log Analytics workspace, I proceeded to configure an Azure Storage account, which would serve as a supplementary data storage component for logs and diagnostic data. Storage accounts in Azure provide scalable and secure cloud storage for a wide range of data types, and in this context, they are essential for persisting certain types of collected telemetry.

From the Azure portal, I once again used the search bar at the top of the interface and typed “**Storage accounts**”. When the blade loaded, I clicked the + **Create** button to initiate the setup of a new storage account.

On the **Basics** tab of the “Create storage account” blade, I configured the required settings. I ensured that the correct **subscription** was selected, and then chose the **AZ500LAB131415** resource group to maintain consistency across all deployed resources. For the **Storage account name**, I entered a globally unique string composed of lowercase letters and numbers, as required

by Azure naming conventions. I ensured that the name was between 3 and 24 characters in length and had not been used previously.

For the region, I once again selected **East US** to remain consistent with the rest of the lab setup. Under **Performance**, I retained the default setting of **Standard**, which provides general-purpose v2 storage optimized for cost-efficiency and latency. Under the **Redundancy** option, I selected **Locally Redundant Storage (LRS)**, which replicates data within a single datacenter to protect against drive failures while minimizing cost.

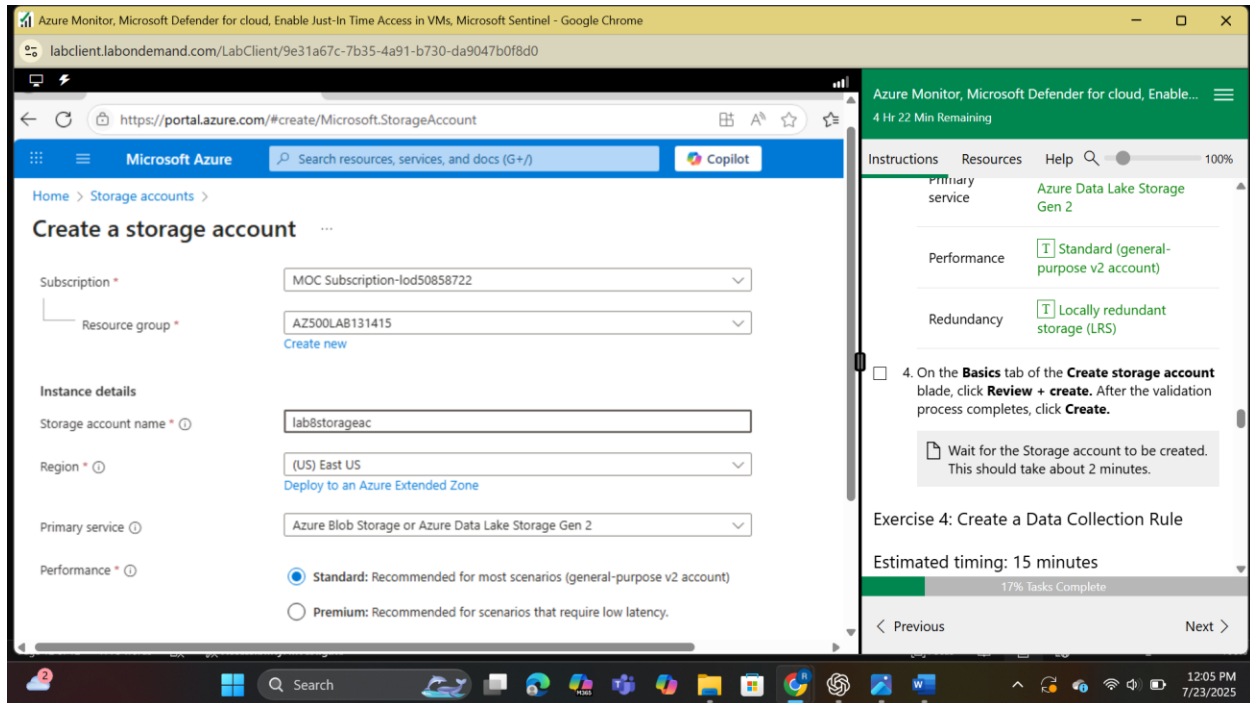


Fig 2.0 Filling in the required details on azure storage account

After configuring these basic parameters, I clicked **Review + create**, and after a successful validation, selected **Create** to begin the deployment. The provisioning process took approximately two minutes, after which the storage account was visible under the list of accounts in the portal, confirming a successful deployment.

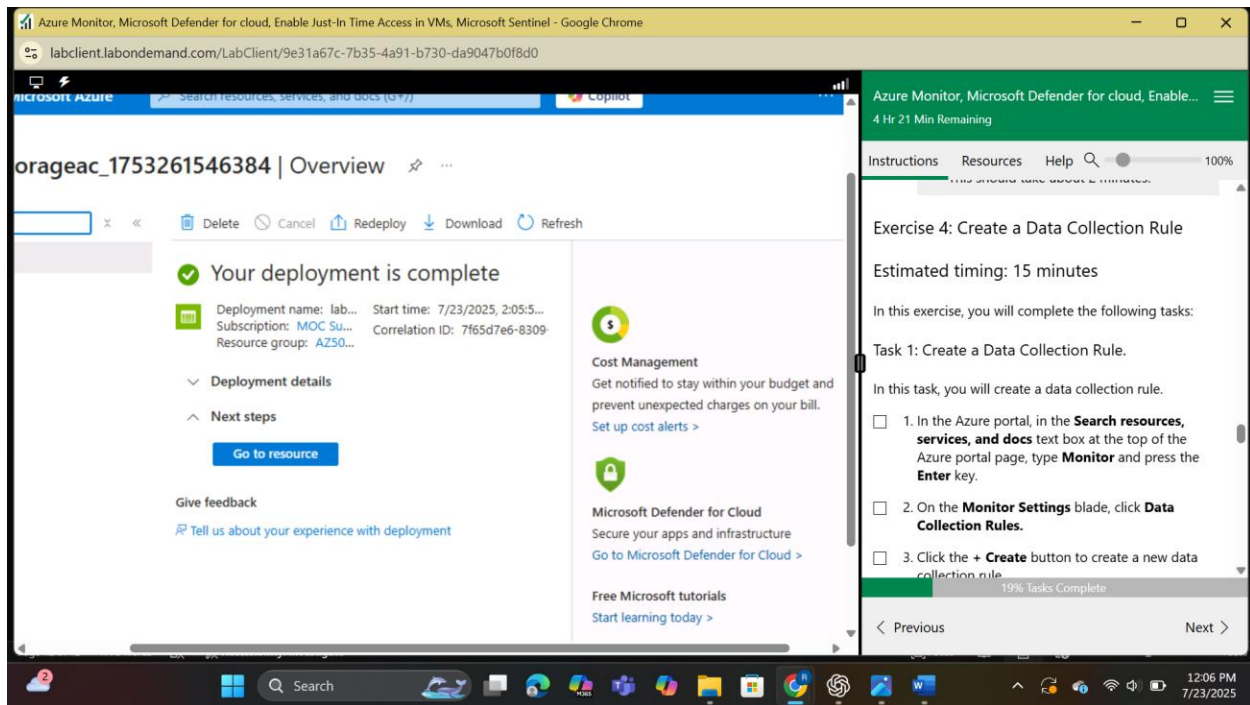


Fig 2.1 Complete deployment of the account

This storage account would later be available for use in diagnostic settings, backup configurations, or additional telemetry collection tasks as needed in subsequent lab exercises. With this step complete, the environment was now equipped with a dedicated workspace and persistent storage layer, both critical components of a robust monitoring and logging solution.

Exercise 4: Creating a Data Collection Rule

Task 1: Creating a Data Collection Rule

With both the Log Analytics workspace and the storage account configured, I moved on to setting up a Data Collection Rule (DCR), which is essential for managing what telemetry is collected from Azure resources and where that data is sent. In this case, the goal was to gather performance counters from virtual machines and route them to the previously created Log Analytics workspace for centralized analysis through Azure Monitor.

To begin, I returned to the Azure portal homepage and used the search bar to look for **“Monitor.”** Once in the Azure Monitor blade, I scrolled through the settings pane and selected **Data Collection Rules**. From there, I clicked on the **+ Create** button to begin configuring a new DCR.

On the **Basics** tab of the “Create Data Collection Rule” wizard, I provided the following configuration details: for the **Rule Name**, I entered **DCR1**. I confirmed the correct **subscription** and chose the existing **AZ500LAB131415** resource group to ensure that the rule would be

created in the same logical boundary as the other lab resources. I set the **Region** to **East US**, matching all previously created components, and selected **Windows** as the **Platform Type**, since the target virtual machine is running the Windows operating system. I left the **Data Collection Endpoint** field blank as instructed.

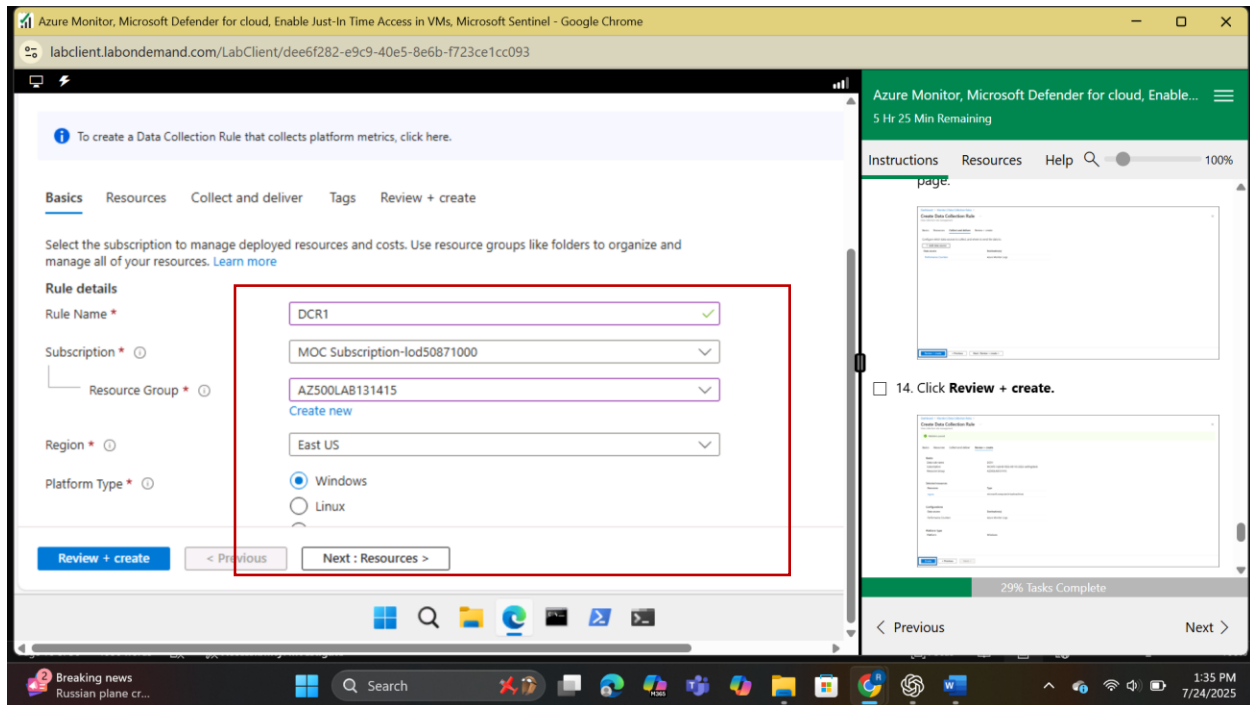


Fig 2.2 Information on the basics blade

With the basic information in place, I clicked **Next: Resources >** to move to the next section. On the **Resources** tab, I selected **+ Add resources**, and in the scope template, I checked the **Subscription** checkbox. After verifying that the subscription scope was correctly set, I clicked **Apply** to proceed.

Next, on the **Collect and deliver** page, I clicked **+ Add data source**, then selected **Performance Counters** from the **Data source type** drop-down menu. I was presented with a list of standard performance counters used in monitoring system health, and I kept the default sample rate of **60 seconds** for each counter type. The included performance counters were **CPU**, **Memory**, **Disk**, and **Network**, all of which are essential metrics for establishing a baseline of virtual machine performance and detecting abnormal behavior.

Once the data sources were configured, I clicked **Next: Destination >**. On this screen, I clicked **+ Add destination**, then selected **Azure Monitor Logs** as the destination type. Under the **Account or namespace** drop-down, I selected the **LAW53202036** Log Analytics workspace created earlier. This binding ensured that all performance data collected under this rule would be routed directly into the Log Analytics environment for querying, dashboarding, and alerting.

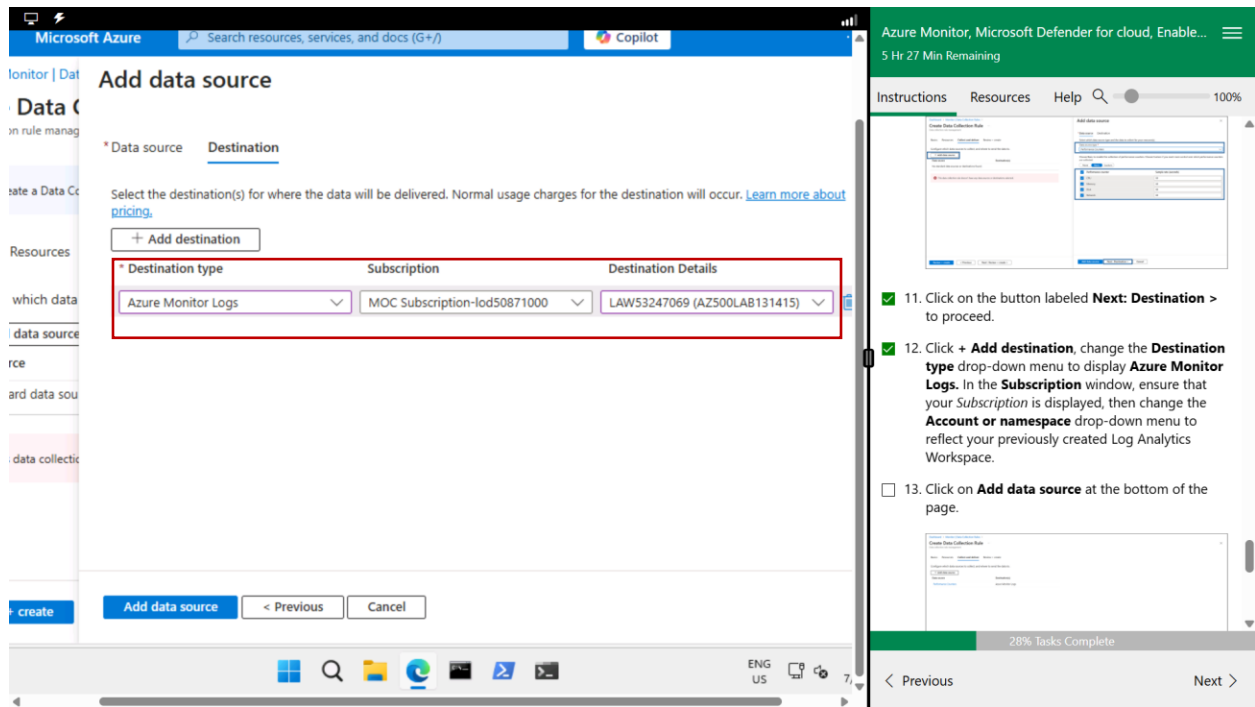


Fig 2.3 Information on the resource blade

I reviewed the configuration on the final tab and then clicked **Create** to deploy the Data Collection Rule. The deployment completed successfully after a brief provisioning period, confirming that the rule was now active and ready to begin collecting telemetry from supported virtual machines.

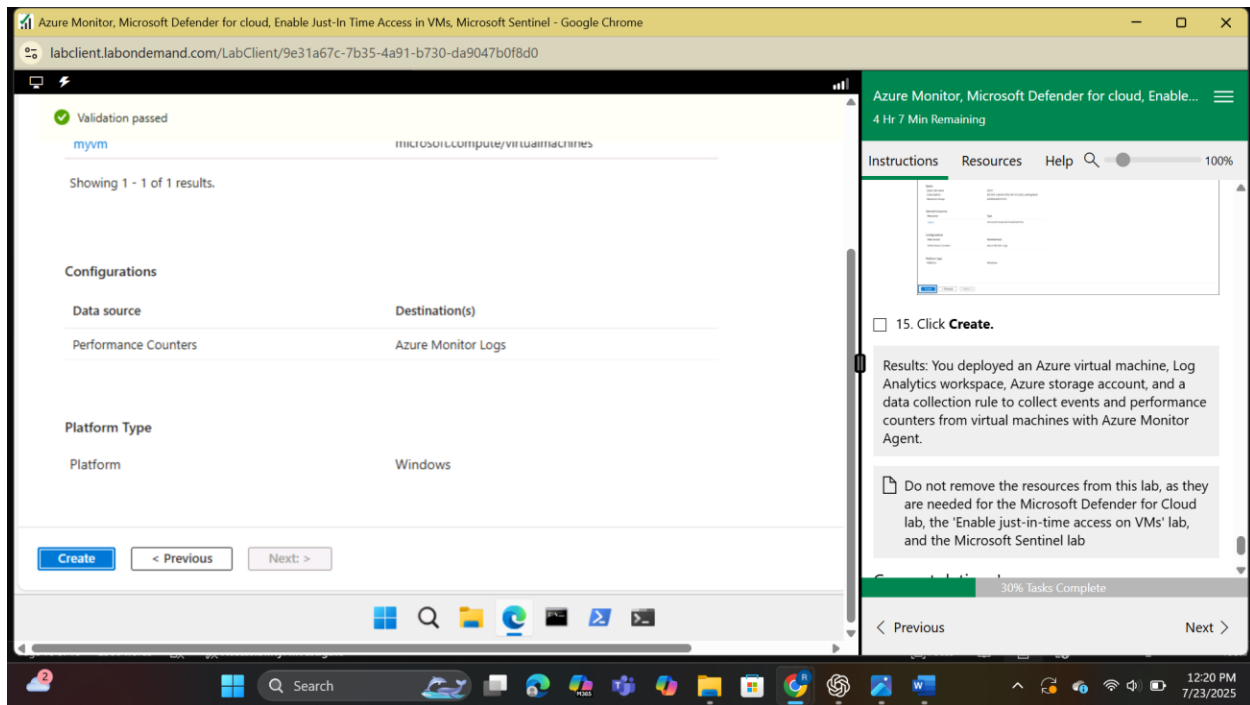


Fig 2.4 Reviewing the configuration and clicking create

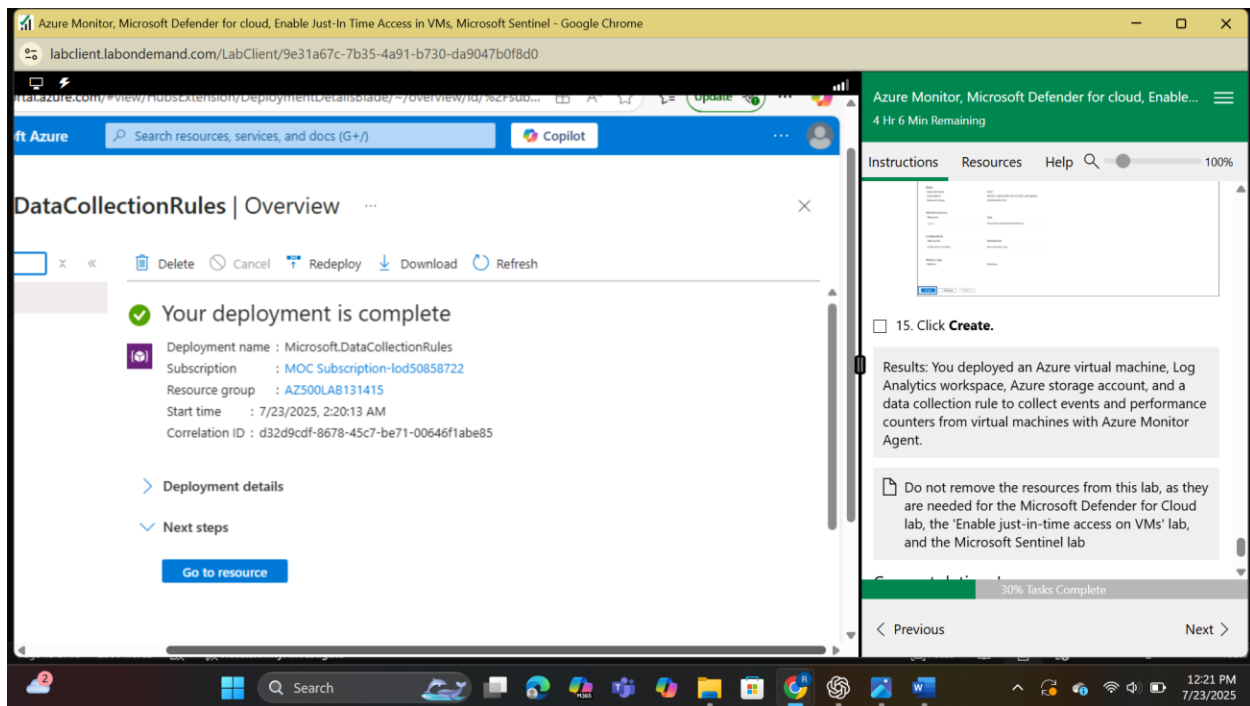


Fig 2.5 Successful deployment of the Microsoft data collection rules

With the DCR now in place, I had established an automated pipeline for collecting, routing, and analyzing performance data—enabling real-time observability and supporting advanced analytics through Azure Monitor. This completed the last configuration task of the lab and fully set up the

monitoring infrastructure needed for secure and reliable VM operations in the Azure environment.

Lab 09: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers

Exercise 1: Configuring Microsoft Defender for Cloud Enhanced Security Features for Servers

To begin the process of enabling enhanced protection for virtual machines, I signed into the Azure portal and accessed **Microsoft Defender for Cloud** by typing the service name into the global search bar at the top of the interface. Once the blade loaded, I navigated to the **Management** section and selected **Environment settings**, which is where security configurations for Azure subscriptions are managed.

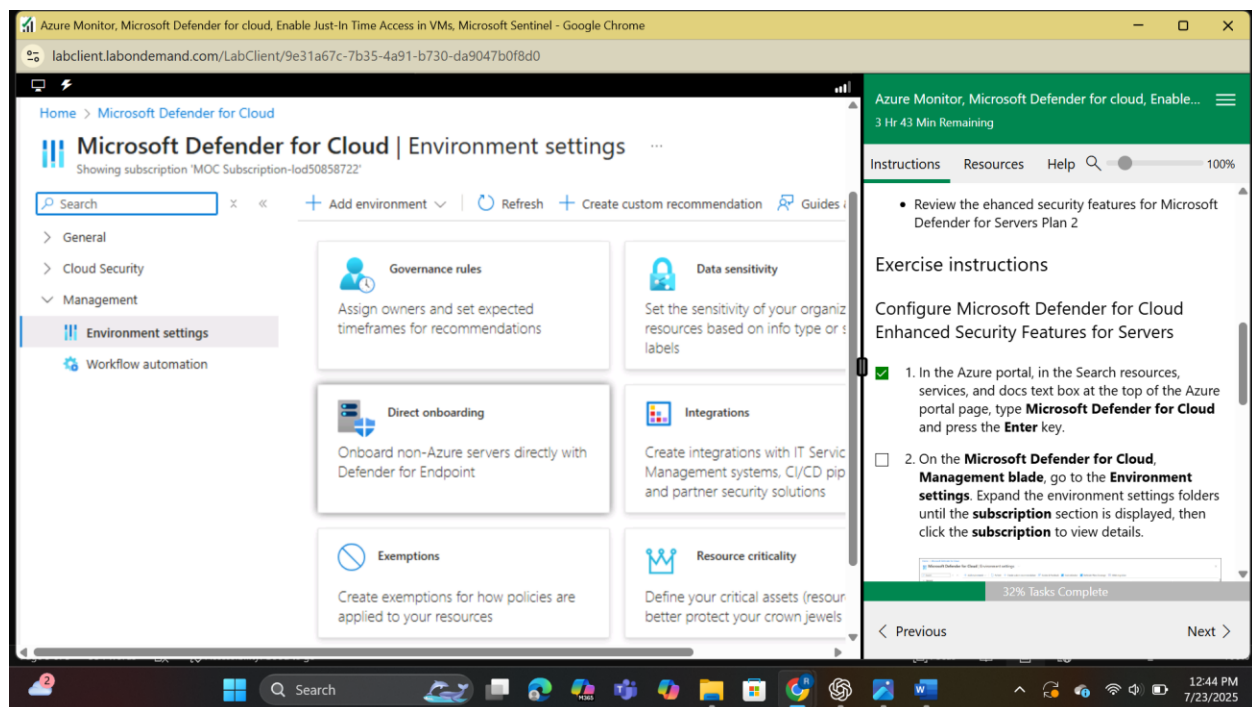


Fig 2.6 The environment setting blade

In the Environment settings pane, I expanded the folder structure until I located and clicked on the subscription used for the lab. This brought up a detailed settings view specifically for that subscription. From there, I located the **Defender plans** section under **Settings**, and expanded the **Cloud Workload Protection (CWP)** category. This section included all available workload-specific Defender plans.

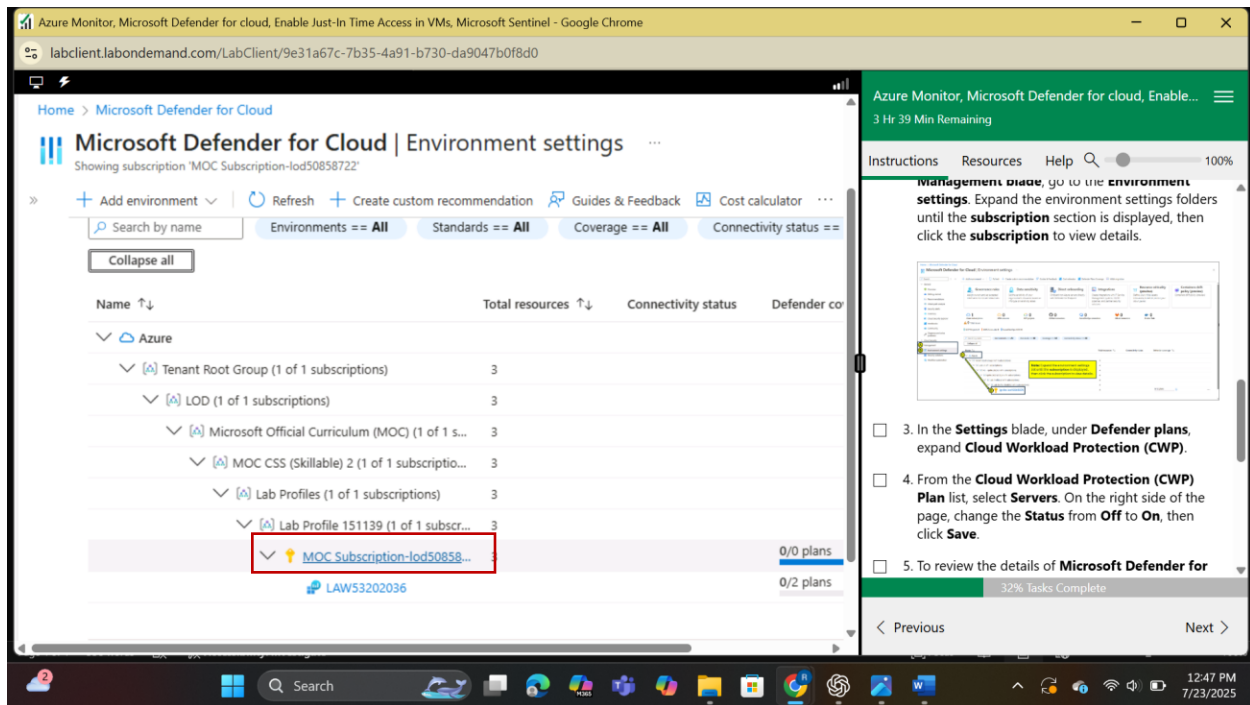


Fig 2.7 Locating the subscription used

Within the CWP list, I selected **Servers**, which opened a configuration panel on the right side of the screen. The current status of the plan was set to **Off**, so I changed it to **On** to activate protection. After toggling the switch, I clicked **Save** to confirm the change. This action enabled **Microsoft Defender for Servers Plan 2**, which provides advanced security features such as threat detection, adaptive application controls, and just-in-time VM access.

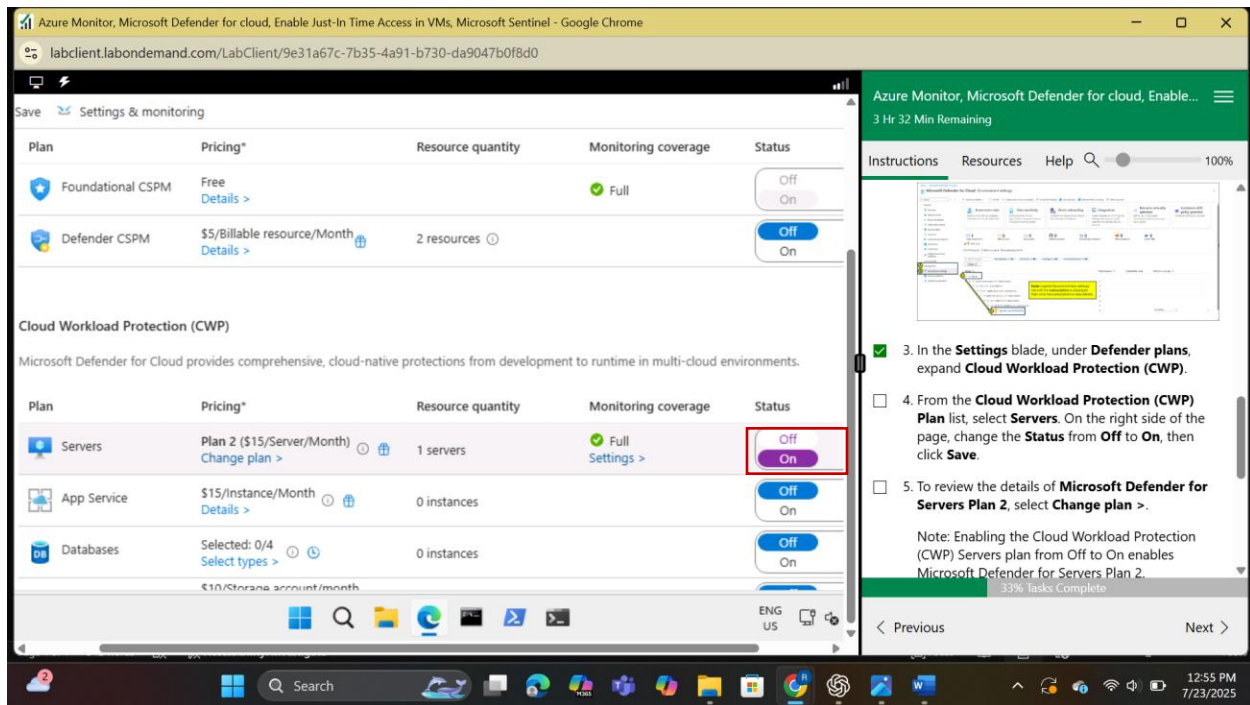


Fig 2.8 Turning the serve status to on

To further review what was included in Plan 2, I clicked on the **Change plan** option. This provided a detailed breakdown of the capabilities enabled by the plan, including integration with Microsoft Defender for Endpoint, file integrity monitoring, and automatic provisioning of the Azure Monitor agent.

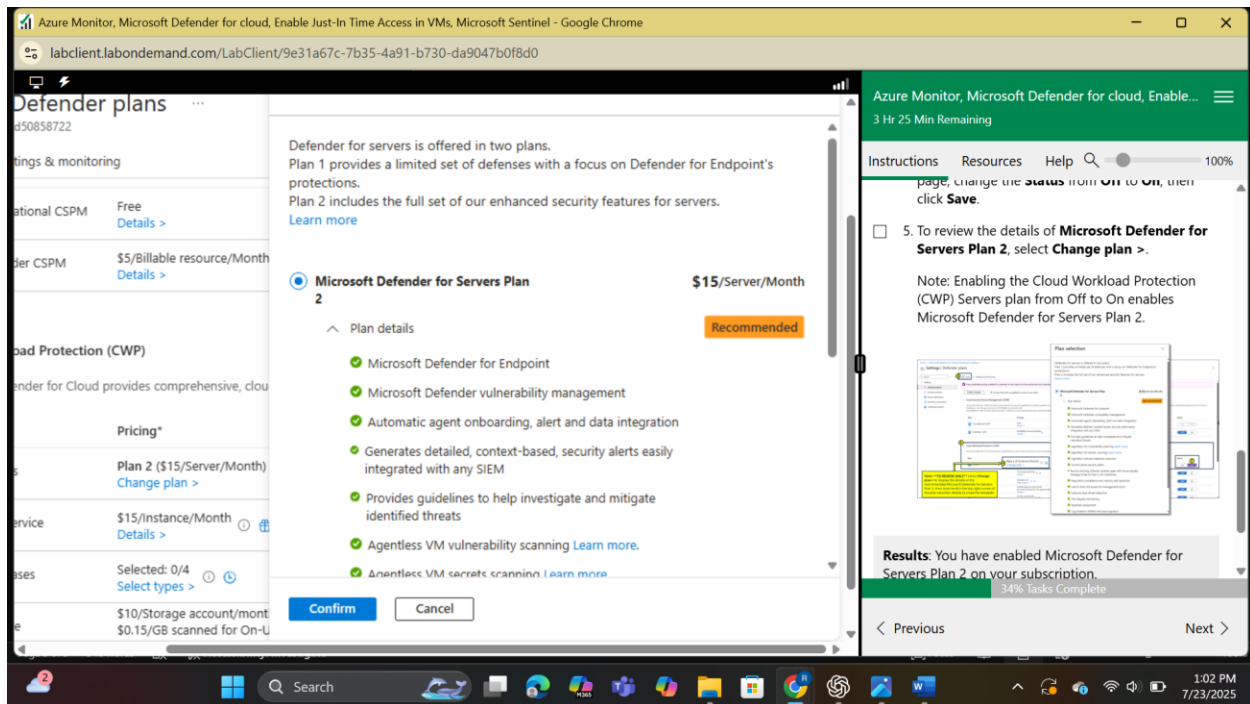


Fig 2.9 Reviewing the details of Microsoft Defender for Servers Plan 2

With this configuration in place, the subscription was now protected by Microsoft Defender for Servers Plan 2, ensuring that all Azure virtual machines under its scope benefit from enhanced threat protection, real-time security alerts, and improved visibility into vulnerabilities and misconfigurations. This completed the lab, successfully enabling a critical layer of defense for hybrid and cloud-based server workloads.